Übungsblatt 4: Race Conditions, Mutex und Priority Inversion

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Zielsetzung

Zwei getrennte Versuche auf dem Cortex-M4 (FreeRTOS über CMSIS-RTOS2; Board wie in Vorlesung 3, **NUCLEO-H755ZI-Q**, sofern das Cube-Projekt das ist). Zuerst schützen Sie eine ganze UART-Nachricht. Danach vergleichen Sie dieselbe CPU-Last einmal mit binärem Semaphor und einmal mit Mutex. Saubere Zeilen sind kein Deadline-Nachweis. Im Kursrepository liegt kein übersetzbares Firmwareprojekt; Pins, Baudrate und IRQ-Prioritäten stehen im jeweiligen Cube-Projekt.

Aufgabe 1: UART, mögliches Fehlbild

1. Zwei Threads, `osPriorityNormal` und `osPriorityHigh`, sofern `cmsis_os2.c` diese Stufen nicht klemmt.
2. Beide schreiben **dieselbe** Ausgabefunktion, ungeschützt, mit `osDelay` **nach** der Ausgabe.
3. Lesen Sie nach, wohin `printf` wirklich geht: Retarget, Puffer, HAL-Polling, IT oder DMA.
4. Notieren Sie die Beobachtung. Möglich sind vermischte Zeichen, `HAL_BUSY`, verlorene Bytes oder bereits serialisierte Bibliothekszugriffe. „Immer Buchstabensalat“ ist kein Soll.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: Ganze Nachricht unter Mutex

```
const osMutexAttr_t uartAttr = { .attr_bits = osMutexPrioInherit };
osMutexId_t uartMx;
```

`osMutexNew` nach `osKernelInitialize`, Handle prüfen. Im genannten FreeRTOS-Adapter bleibt PIP oft auch ohne das Bit aktiv; das Bit dokumentiert die Absicht und schaltet PIP dort nicht zuverlässig ab.

Ablauf je Thread: in einen privaten Puffer formatieren, Acquire, gesamte Ausgabe inklusive Abschluss, Release, danach erst `osDelay`. Acquire-Fehler: nichts schreiben und nichts freigeben. `osDelay` nicht im gehaltenen Mutex.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Inversion mit Start-Gate

Prioritäten prüfen: L normal, M `osPriorityAboveNormal`, H `osPriorityHigh`. Idle bleibt darunter.

Der Test-Lock ist ein binärer Semaphor, anfangs verfügbar:

```
osSemaphoreId_t testSem;
osSemaphoreId_t gateH;
osSemaphoreId_t gateM;
```

`testSem = osSemaphoreNew(1, 1, NULL)`, die Gates `osSemaphoreNew(1, 0, NULL)`. Gates und Test-Lock nicht vermischen.

1. H und M blockieren zuerst an ihrem Gate.
2. L nimmt `testSem`, setzt ein sichtbares „Lock gehalten“ (GPIO oder RAM-Protokoll, kein `printf` in der Messung).
3. Erst danach Releases der Gates. H ist höher und soll den belegten Lock zuerst anfordern.
4. L arbeitet danach ein **begrenztes** CPU-Pensum weiter, ohne `osDelay`. Das Pensum muss lang genug sein, dass H den Acquire versucht, während L noch Owner ist. `volatile` allein legt die Dauer nicht fest; Ergebnis benutzen und die Iterationen protokollieren.
5. M arbeitet ein begrenztes Pensum, fasst den Test-Lock nicht an und blockiert dann.
6. Eine Endlosschleife von M nur als separaten Grenzfall: nach dem Release kann L weiter verhungern. PIP ist dann keine Fairness.

Nicht `osDelay` als Ersatz für die CPU-Last von L verwenden. Eine Variante mit blockierendem UART wäre ein anderes Experiment: Vererbung beschleunigt keine noch laufende Übertragung.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Dieselbe Last mit Mutex

Tauschen Sie Typ und alle Aufrufe, nicht nur den Namen:

```
const osMutexAttr_t testAttr = { .attr_bits = osMutexPrioInherit };
osMutexId_t testMx;
/* osMutexNew, osMutexAcquire, osMutexRelease */
```

Dieselben Gates, dieselben `cpu_work`-Argumente, dieselbe Freigabereihenfolge. Gates bleiben Semaphoren.

Messen Sie L-Acquire, H-Anforderung, M-Anfang und M-Ende, L-Release, H-Erfolg, H-Ende. `printf` erst danach. Gemeinsame Logs brauchen einen eigenen Schutz. `CYCCNT` nur bei laufendem Zähler und bekanntem Takt; er enthält auch fremde Zeit.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)